22nd Aug Unit 1

Operators in C programming

1) NOT (!)

It is the property of not operator that not of true is false and not of false is true.

$!T = F$

$!F = T$

In C, C++ or Java all +ve and all -ve numbers are true and only zero is considered as false.

Ex - 1-1000 = 0

!!!1000

0

1 = 0

2) Relational Operator

<, >, <=, >=, ==

P(5 > 2) ⇒ 1

P(5 > 3 > 2)
1 > 2 ⇒ 0

= (Assignment) == (Comparison)

- Basic format specifies in C programming :

1 %d → Integer = int 1, 2, 3, -4
2 %c → characters = char 'a', 'b', 'c'
3 %f → floats = float 2.5, 3.5
4 %s → strings : Array of characters "a", "b", "c"

- int x;
  int y; or int x, y, z;
  int z;

If int x; y; z; // compiler error

- int x, y, z; → variable declaration
  x = 10; → Initialization
  y = 5;
  z = x+y;
  p(z)

Assign value

- int x;
  x=5;
  x=6;
  p(x)
  Output :
  x
  ~~5~~ 6

- int x;
  x=5;
  x+1; → Not assign
  p(x)

  x=5

- int x=5;
  p("%d %d", x=5, x==6);
  Output:
  5 0

→ In comparison operator if both the number are same it return one otherwise it return zero.

→ Example: int x, y, z;
x=5;
y=3;
z=(x=y);
Output : x=3
y=3
z=3

→ ! (Unary) require 1 operant
Relational (Binary) require 2 operant

- Modulus Operator (%)

It gives remainder.

Example → 5%2 = 1

-5%2 = -1

-5%-2 = -1

5%-2 = 1

5.2%2 = Error

It is the property of modulus operator that is always takes the sign of first operant.

We cannot apply modulus to a floating type value. It can only be applied onto the integer. It is a binary operator.

VV Imp ~~Logical &&~~ Logical AND && and Logical OR ||

&&			||		
T	T	T	F	F	F
F	T	F	T	T	T
F	F	F	T	F	T
T	F	F	F	T	T

→ int x, y, z;

x=5, y=2;

z = x = 0 && y = 4;

F T

F

z = 0

y = 2

x = 0

Green House
Date
Page No.

* In case of logical AND, if first expression is false compiler will not evaluate the 2nd expression. But if first expression is true then the compiler will evaluate the 2nd expression.

$x = 5, y = 2;$
$z = x = 6 \&\& y = 0$
T

Output : $x = 6$
$y = 2$
$z = 1$

* In case of Logical OR, if first expression is true compiler will not evaluate the 2nd expression. But if first expression is false then compiler will evaluate the 2nd expression.

→ Priority of Logical AND is higher than Logical OR

Example :- int x, y, z;
$x = 2, y = 0;$
$z = (y \&\& x) || y = 5;$
$x = 20$
F F T
F T
T

Output :
$x = 2, y = 5, z = 1$

• int x, y, z, k;
$x = -1, y = 0, z = 2;$
$k = x == 0 || (y = -2 \&\& z = 3);$
F T T
F T
T

Output :
$k = 1, x = -1$
$y = -2, z = 3$

Green House
Date
Page No.

- Increment and Decrement

++	--
++x (pre)	--x (pre)
x++ (post)	x-- (post)

++x, x++ > x = x + 1

--x, x-- > x = x - 1

- int x = 5;
  x++;
  p(x)
  Output:
  6

- int x = 5;
  ++x;
  p(x)
  Output:
  6

- int x = 5, y;
  y = x++;
  p(x, y);
  Output:
  6, 5

  y = x; x = x + 1;

- int x = 5, y;
  y = ++x;
  p(x, y)
  Output:
  6, 6

  y = x = x + 1

→ Post increment means when the same variable is encountered in the program again then the value will be increased.

- int x, y, z;
  x = 2, y = 3
  z = x++ + x-- - --y + y++ - --x;
  2*+ 3*- 2 2*+ 1
  Output: 2 + 3 - 2 + 2 - 1
  4
  x = 1
  y = 3
  z = 4

Date
Page No.

- int x, y, z, k, l;
  x = 2, y = 3, z = 2;
  k = x++ - y--- + --z;
  $\quad 2^{*+} \quad 3^{*-} \quad 1$
  l = k++ - --z + y++ + --x;
  $\quad 0^{*+} \quad 1 \quad 3^{*-} \quad 2$

  Output:
  x = 2, y = 3, z = 0, k = 1, l = 4

- int x, y, z, k, l;
  x = 2, y = 3, z = 1, k = 0;
  l = (k++ && --z) || (y++ && --x);
  $\quad 0^{*+} \quad 0 \quad 3^{*+} \quad 1$
  $\quad F \quad T$

  Output:
  x = 1, y = 4, z = 1, k = 1, l = 1

- int x, y, z, k, l, m,
  x = -1, y = 1, z = 0, k = 2, l = -1;
  m = l++ || (--x && --y) || (++z && k++) || --l;
  $\quad -1^{*-} \quad -2 \quad 3 \quad 2 \quad 1^{*-} \quad 0$
  $\quad T \quad T \quad T \quad T \quad T \quad F$
  not to solve.

  Output:
  x = -1 z = 0 l = 0
  y = 1 k = 2 m = 1

- int x, y, z, k, l, m;
  x = -1, y = 1, z = 0, k = 2, l = -1
  m = (++l && --x) && ++y || z++ || k--;
  $\quad 0 \quad F \quad \times \quad 0^{*+} \quad F \quad 2^{*-}$

  Output: x = -1 z = 1 l = 0
  y = 1 k = 1 m = 1

Green House
Date
Page No.

- int x, y, z, k, l, m;

  x = -1, y = 1, z = 0, k = 2, l = -1;

  m = x++ || (--y && --z) || (++k && ++l);

  -1 ++ 0 3 0

  T F not to solve T F

  F

Output:

x = ~~1~~ 0 z = 0 l = ~~2~~ -1

y = ~~0~~ 1 k = ~~3~~ 2 m = 1

- Arithmetic Assignment:

  +=, /=, *=, %=, -=

  int x = 5;

x += 2;	x = x + 2	7
x /= 2	x = x / 2	3
x *= 3	x = x * 3	9
x %= 2	x = x % 2	1
x -= 2	x = x - 2	-1
	x = -1	

- Unary → We take only one operant.

  Eg → ++x, x++, --x, x--, !, Unary+, Unary-

  Binary → We take two operant.

  Eg → <, >, <=, >=, ==, %, +, /, +=.

- Ternary Operator → It involves three operant. Denoted by (?)

  Eg → int x = 5

  x == 5 ? P("X") : P("Y");

  I II III

I → True → II
I → false → III
Output : X

- #include < stdio.h > → Standard Input Output
  - ↓ pre-processor directive.
  - scanf() ↳ printf()

Before processing the code it sends a message to the compiler to include the header file <stdio.h>.

- #include < conio.h > → console input output

// getch() → It hold the output on the screen.

- main () → Starting point of program
- void main () → function
  - ↓ Return type
  - It means null or nothing
- { ... } → Scope of the program

```
{ int x, y, z ; → declaration of variable
  printf ("Enter two no's ");
  scanf ("%d %d", &x, &y);
  z = x + y;
  printf("%d", z);
}
```

Output
Input

→ reference operator or address operator
↳ gives address of operator.

Q- Write a program to swap two numbers using a third variable.

```
#include<stdio.h>
#include< conio.h>
void main()
{
```

x	y	z
20 [10]	10 [20]	[10]
1000	2000	3000

x = 10
y = 20

Date
Page No.

```
int x,y,z;
printf("Enter the two numbers");
scanf("%d%d",&x,&y);
z=x;
x=y;
y=z;
printf("x=%d And y=%d",x,y);
getch();
}
```

pf("x"); x
pf("%d",x); 10
pf("%d",&x); 1000
↓ Address operator

Output : $\boxed{x=20 \text{ And } y=10}$

**Q- Write a program ~~two~~ to swap two no.s without using third value.

```
#include <stdio.h>
#include <conio.h>
void main()
{
 int x, y;
 printf("Enter two numbers");
 scanf("%d%d",&x,&y);
 x=x+y;
 y=x-y;
 x=x-y;
 printf("x=%d, y=%d",x,y);
 getch();
}
```

x: 20 ~~30~~ $\boxed{\cancel{10}}$
y: $\boxed{\cancel{20}}$ 10

Q- Write a program to find the average marks of five subject

```
#include <stdio.h>
#include <conio.h>
void main()
{
  float Maths, Bio, Eng, Chem, CS, sum, avg;
  printf("Enter the marks of 5 subjects");
  scanf("%f %f %f %f %f", &Maths &Bio &Eng &C
  sum = Maths + Bio + Eng + Chem + C.S
  Avg = sum/5;
  printf("%f", Avg);
  OR print("%f", (Maths+Bio+Eng+Chem+CS)/5);
  getch();
}
```

Better way

- printf function : a) It is the inbuilt output function in c which prints the output onto the screen.
  b) Printf() returns no. of characters.
  c) We can have nested printf also.

Example : • printf("WHY");

Output : WHY

• x = printf("WHY");
  printf("%d", x);

Output : WHY
3

• x = printf("%d/n", printf("LOVE/n"));
  printf("%d", x);

4 → But this is 1 character

Output : LOVE
4
1

→ for newline : /n

1

• $x = \text{printf}(\text{"\%d"}, \text{printf}(\text{"\%d"}, \text{printf}(\text{"HATE"})))$
printf("%d", x);

4 4

Outputs: | HATE 4 1 1 |

m&CS): Scanf ( ): a) It is the inbuilt input function in C which takes input from the keyboard.
b) Scanf ( ) returns no. of arguments.
c) no. of arguments is denoted by no. of formats specified.

Example: • y = scanf("%d", &x); // x = 1000
printf("%d %d", x, y); x = 1,00,000
x = 1 Lakh
x = 0

Output: | 1000 1 |
| 1crore 1 |
| 1 lakh 1 |
| 0 1 |

1 1 return no. of character

• $y = \text{printf}(\text{"\%d"}, \text{scanf}(\text{"\%d"}, \&x))$; // x = 0
printf("x = %d, y = %d", x, y);

Output: | 1 0 1 |

• printf("Enter two No.'s");
scanf("%d pqr %d", &x, &y); // x = 10 y = 20
printf("x %d y = %d", x, y);

| 10pqr20 |
↓
Input to be given in proper formats as given.

Output: | 10 20 |